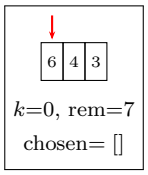
Lecture Slip, CS 235
Oct 29, 2026, Thursday
Pruning the Search Tree

Name:	
EmpID:	

1. **Pruning and early termination.** Here is `find` from class. It asks whether some subset of `v[k..]` adds up to `rem`.

```
bool find(const std::vector<int>& v, int k, int rem, std::vector<int>& chosen) {
  if (rem == 0) { return true; }                                  // goal
  if (k == static_cast<int>(v.size()) || rem < 0) { return false; } // dead end
  chosen.push_back(v[k]);                                         // choose
  if (find(v, k + 1, rem - v[k], chosen)) { return true; }        // take
  chosen.pop_back();                                              // undo
  return find(v, k + 1, rem, chosen);                             // skip
}
// v = {6, 4, 3}; chosen starts empty; first call: find(v, 0, 7, chosen)
```

For the recursive call tree in part (c), draw each call as a **node** like the example below: show the array `v` with an arrow at `v[k]`, followed by `k`, `rem`, and `chosen`.

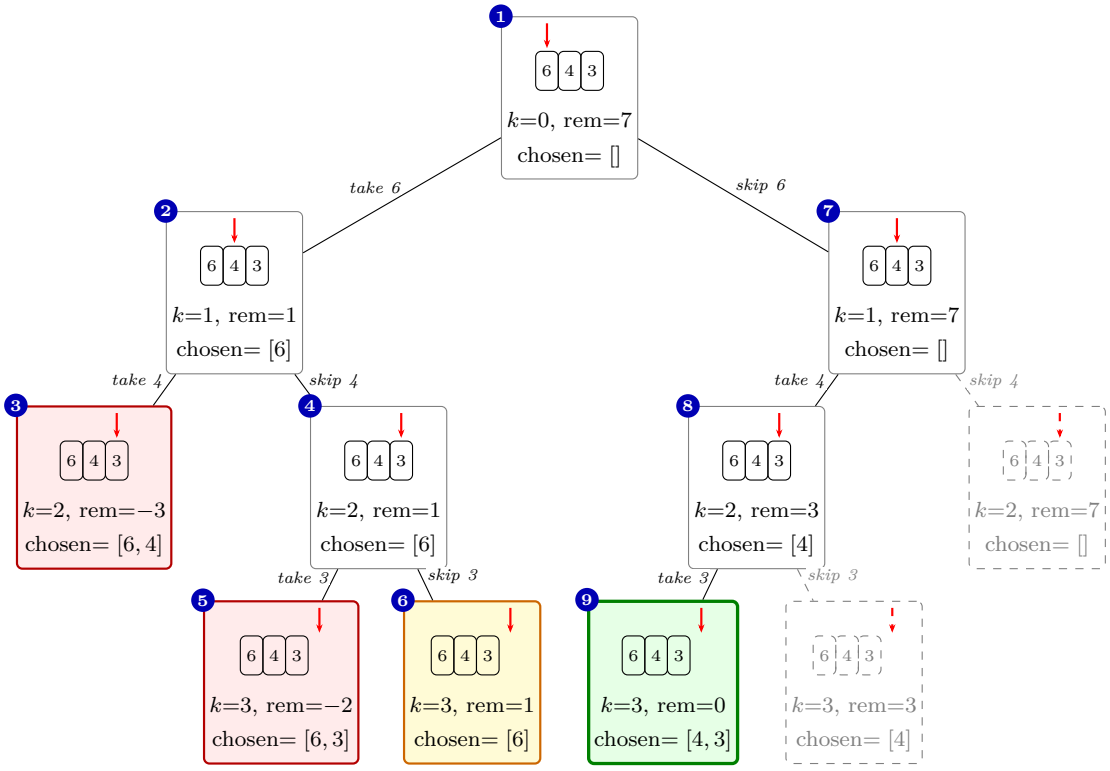


(a) List every call in order as a triple (`k`, `rem`, `chosen`). How many calls are there?

(b) Which calls are stopped because `rem < 0`? What is `chosen` when the first call returns?

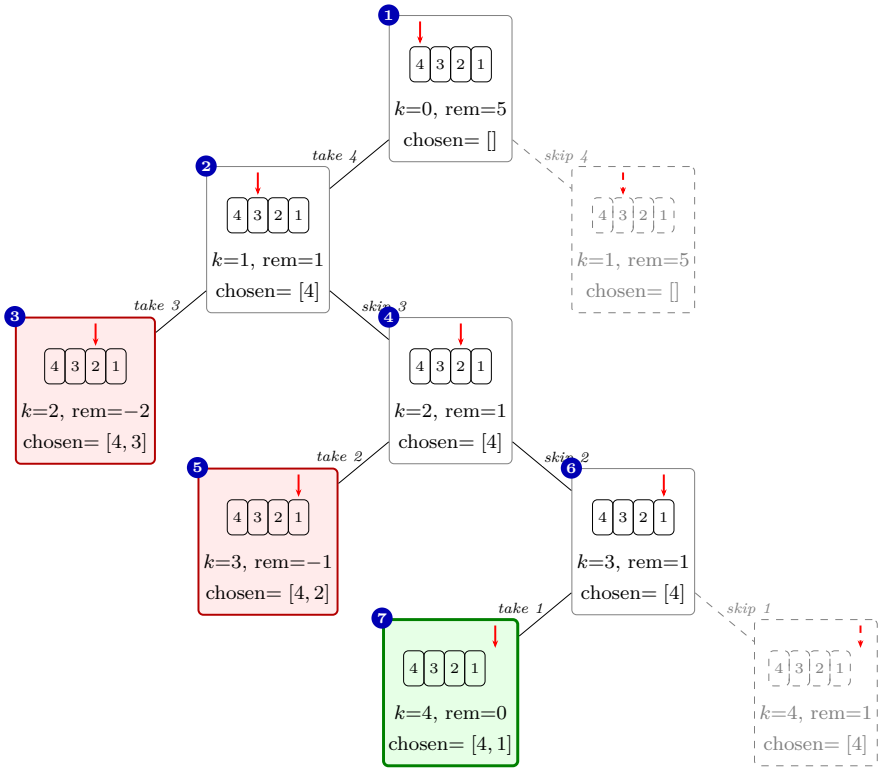
(c) Now let `v = {4, 3, 2, 1}` and make the first call `find(v, 0, 5, chosen)`. Draw the recursive call tree with nodes as above: label each edge take or skip, and number the calls in order. How many calls are there, and what is `chosen` when the first call returns?

Answer: (a) $(0, 7, [\,])$, $(1, 1, [6])$, $(2, -3, [6, 4])$, $(2, 1, [6])$, $(3, -2, [6, 3])$, $(3, 1, [6])$, $(1, 7, [\,])$, $(2, 3, [4])$, $(3, 0, [4, 3])$: **9 calls**. (The full take/skip tree has 15 nodes.)



(b) $(2, -3, [6, 4])$ and $(3, -2, [6, 3])$ are stopped by `rem < 0`: taking 6 and then 4 overshoots, and so does 6 and then 3. The call $(3, 1, [6])$ is a different kind of dead end: no numbers are left and `rem` is not 0. Early termination: after the goal $(3, 0, [4, 3])$, every call returns `true`, and `chosen` is `{4, 3}`, since $4 + 3 = 7$. The skip branches that were still waiting (gray, dashed) are never run.

(c) **7 calls**; `chosen` is `{4, 1}`, since $4 + 1 = 5$. The calls numbered 3 and 5 are stopped by `rem < 0`. The subset $\{3, 2\}$ also adds to 5, but the search takes before it skips, so it reaches $\{4, 1\}$ first, and then stops: early termination again (the dashed gray nodes are never created).


1
4 3 2 1
k=0, rem=5
chosen= []
take 4
skip 4
2
4 3 2 1
k=1, rem=1
chosen= [4]
4 3 2 1
k=1, rem=5
chosen= []
take 3
skip 3
3
4 3 2 1
k=2, rem=−2
chosen= [4, 3]
4
4 3 2 1
k=2, rem=1
chosen= [4]
take 2
skip 2
5
4 3 2 1
k=3, rem=−1
chosen= [4, 2]
6
4 3 2 1
k=3, rem=1
chosen= [4]
take 1
skip 1
7
4 3 2 1
k=4, rem=0
chosen= [4, 1]
4 3 2 1
k=4, rem=1
chosen= [4]

2. **A bound.** Let `v = {5, 4, 3, 2, 1}` and `n = v.size()`. Define `suffix[k]` as the sum of `v[k..4]`, with `suffix[5] = 0`. The pruning test `rem > suffix[k]` is added, so the dead-end line becomes `if (k == n || rem < 0 || rem > suffix[k]) return false;`

   (a) Write the array `suffix[0..5]`.

   (b) With target 20, how many calls does the search make? With target 9, how many calls, and what is `chosen`?

   Answer: (a) `suffix = {15, 10, 6, 3, 1, 0}`.

   (b) Target 20: **1 call**. At the first call, `rem` $= 20 >$ `suffix[0]` $= 15$: even taking every number cannot reach 20, so the search returns `false` at once. Target 9: **3 calls**: $(0, 9)$, then take 5 gives $(1, 4)$, then take 4 gives $(2, 0)$, the goal. `chosen = {5, 4}`.

3. **Counting Make 24 moves.** In class, a move picks an **unordered pair** and replaces it by one result, with 6 possible results per pair $(a+b,\ a\times b,\ a-b,\ b-a,\ a\div b,\ b\div a)$. For counting purposes, assume all six operations are available at every step.

   (a) With **5 numbers** on the table, how many first moves are there?

   (b) Starting from 5 numbers, how many complete sequences of moves are possible under this assumption? Write the product.

   Answer: (a) $\binom{5}{2} \times 6 = 10 \times 6 = \mathbf{60}$ first moves.

   (b) The numbers on the table are 5, 4, 3, 2, and each level multiplies: $60 \times (6 \times 6) \times (3 \times 6) \times (1 \times 6) = 60 \times 36 \times 18 \times 6 = \mathbf{233\,280}$. (For four numbers it was $36 \times 18 \times 6 = 3888$: one more number multiplies the search by 60.) Under the stated assumption (all six operations always available) this count is exact. A real solver that skips division by zero explores fewer sequences, so for it 233 280 is an upper bound.

4. **Duplicate letters.** `printUnique` from class skips a letter that was already tried at the same position. Let `std::string s = "LEVEL";` be the input.

   (a) How many lines would the original `printAll(s, 0)` print? How many does `printUnique(s, 0)` print?

   (b) Why does the original print so many more lines?

   Answer: (a) `printAll`: $5! = \mathbf{120}$ lines. `printUnique`: $5!/(2!\,2!) = \mathbf{30}$ lines (the letters `L` and `E` each appear twice; `V` once).

   (b) `printAll` treats the two `L`s as different letters, and the two `E`s too, so each distinct arrangement appears $2 \times 2 = 4$ times ($120 = 30 \times 4$). `printUnique` tries each letter value only once per position, so repeated branches are never explored.

5. Do you have any questions about today's class? If so, please provide details. Thank you.